



Lista de Exercícios 3

8 de abril de 2022

Engenharia de Computação

Lilian Giselly Pereira Santos

Pedro Henrique de Brito Nascimento

Projeto e Análise de Algoritmos - Rian Gabriel Pinheiro

Capítulo 3 - Divisão e Conquista

1. Existem n panquecas, todas de tamanhos diferentes, empilhadas umas sobre as outras. Você pode colocar uma espátula sob uma das panquecas e virar a pilha inteira acima da espátula. O objetivo é arranjar as panquecas de acordo com o tamanho, com a maior na parte inferior. A Figura mostra uma instância do quebra-cabeça para $n = 7$. Projete um algoritmo para resolver este problema e determine o número de operações feitas pelo algoritmo no pior caso.



Solução:

```

//considere V a pilha de panquecas e n a quant. de panquecas q ainda não foram ordenadas
//função para inverter a pilha inteira acima da maior
int flipPancakes(int *V, int n){
    int start=1, end=n, aux, i;
    while(start < end){ // percorre todo o vetor invertendo as posições
        aux = V[start];
        V[start] = V[end];
        V[end] = aux;

        start++;
        end--;
    }
}

//função para descobrir qual a posição da maior panqueca da pilha
int biggerPancake(int *V, int n){
    int i = 1, bigger = 1;
    for (i=1; i<n; i++){
        printf("%d - ", V[i]);
        if(V[i] > V[bigger]) bigger = i; // percorre todo o vetor e
        // atualiza a variável bigger
    }
    printf("A maior panqueca esta na pos: %d", bigger);
    return bigger;
}

void pancakes(int *V, int n){
    if(n == 1){
        printf("\nTodas as panquecas foram empilhadas!\n");
        return;
    }

    int posBigger = biggerPancake(V, n);
    flipPancakes(V, posBigger); //primeira inversão com tds acima da
    //maior panqueca, deixa a maior no topo

    flipPancakes(V, n-1); // inverte td a pilha, a maior panqueca
    // encontrada nessa iteração fica na posição n-1

    pancakes(V, n-1);
}

void main(){
    int V[8] = {0, 2, 3, 1, 6, 4, 5, 7}; // começa com as panquecas
    // empilhadas como na figura
    pancakes(V, 7);

    int i;
    printf("Essa eh a pilha atual de panquecas:\n");
    for (i=1; i<=7; i++)
        printf("%d - ", V[i]);
}

```

No pior caso, temos que o número de operações feitas pelo algoritmo será $O(n^2)$, uma vez que, no pior dos casos, o algoritmo executa $2(n - 1)$ operações de virar a pilha e cada uma destas custam tempo linear.

2. Suponha que você tenha os resultados de um torneio concluído no qual n equipes jogaram entre si uma vez. Assumindo que não houve empate, mostre um algoritmo que liste os times em uma sequência de forma que todos ganhem o jogo com o time listado imediatamente após?

Solução:

Neste caso, como houveram apenas uma partida entre as equipes e assumindo que não houve empate, é suficiente listar em sequência um vetor formado exclusivamente de times ganhadores, nos quais os elementos desse vetor são pares (x_i, y_i) , sendo x_i o time ganhador e y_i o time perdedor, de modo que:

Seja $A[a_0, a_1, \dots, a_{\lfloor \frac{n \cdot (n-1)}{2} \rfloor}]$, um vetor de tamanho $\lceil \frac{n \cdot (n-1)}{2} \rceil$ (dado pela combinação de n times nC_2). Supondo também que a entrada mostre apenas os R resultados possíveis, dados pela combinação anterior, e em sequência na ordem dos times (*e.g.* : $n = 4 \Rightarrow R_{(1,2)}, R_{(1,3)}, R_{(1,4)}, R_{(2,3)}, R_{(2,4)}, R_{(3,4)}$). É possível implementar o seguinte algoritmo:

```
#include<vector>
#include<iostream>
using namespace std;

int main()
{
    vector< pair <int,int> > equipes; // Vetor de pares informando
    // equipe ganhadora e perdedora
    int n, aux; // Numero de Equipes
    cin >> n;
    aux = n;

    while(--aux){
        int partidas = aux;
        while(partidas--){
            int x,y;
            cin >> x >> y;
            if(x>y) equipes.push_back(make_pair(n-aux,n-partidas));
            else equipes.push_back(make_pair(n-partidas,n-aux));
        }
    }
    n = n*(n-1)/2; //Combinações de jogos possíveis

    for (int i=0; i<n; i++)
        cout << equipes[i].first << " " << equipes[i].second << endl;

    return 0;
}
```

3. Escreva um algoritmo de divisão-e-conquista ($n \log n$) para computar a^n em que n é um inteiro positivo.

Solução:

```
long fast_exp(int a, int b)
{
    if(b==0) return 1; // Caso base, expoente = 0
    long result = fast_exp(a,b/2); // Computa a^([b']/2) em log (b'),
    // onde b' é o expoente para este subproblema
    result *= result; // Computa a^([b']/2)* a^([b']/2), tempo constante

    if(b&1) result *=a; //Se expoente ímpar, corrige o resultado
    //e.g.: b=3 => a^b = a^3 = a * a^2
    return result;
}
```

4. São dadas duas listas ordenadas de tamanho m e n . Dê um algoritmo de tempo $O(\log m + \log n)$ para computar o k -ésimo menor elemento da união das duas listas.

Solução:

```
int k_esimo_menor(int *lista_A, int *lista_B, int *fim_lista_A, int *fim_lista_B, int k)
{
    if (lista_A == fim_lista_A)
        return lista_B[k];
    if (lista_B == fim_lista_B)
        return lista_A[k];

    int meioA = (fim_lista_A - lista_A) / 2;
    int meioB = (fim_lista_B - lista_B) / 2;
    if (meioA + meioB < k)
    {
        if (lista_A[meioA] > lista_B[meioB])
            return k_esimo_menor(lista_A, lista_B + meioB + 1, fim_lista_A, fim_lista_B,
                                   k - meioB - 1);
        else
            return k_esimo_menor(lista_A + meioA + 1, lista_B, fim_lista_A, fim_lista_B,
                                   k - meioA - 1);
    }
    else
    {
        if (lista_A[meioA] > lista_B[meioB])
            return k_esimo_menor(lista_A, lista_B, lista_A + meioA, fim_lista_B, k);
        else
            return k_esimo_menor(lista_A, lista_B, fim_lista_A, lista_B + meioB, k);
    }
}
```

5. Suponha que esteja escolhendo entre os seguintes três algoritmos:

- Algoritmo A resolve problemas dividindo-os em cinco subproblemas de metade do tamanho, solucionando cada subproblema recursivamente e, então, combinando as soluções em tempo linear.
- Algoritmo B resolve problemas de tamanho n resolvendo recursivamente dois subproblemas de tamanho $n - 1$ e, então, combinando as soluções em tempo constante.
- Algoritmo C soluciona problemas de tamanho n dividindo-os em nove subproblemas de tamanho $n/3$, resolvendo recursivamente cada subproblema e, então, combinando as respostas em tempo $O(n^2)$.

Qual o tempo de execução de cada um desses algoritmos (em notação O) e qual você escolheria?

Solução:

Utilizando o Teorema Mestre para os algoritmos A e C e resolvendo a recorrência para o algoritmo B, temos que:

$$T_A(n) = 5 \cdot T\left(\frac{n}{2}\right) + O(n) \Rightarrow T_A(n) = O(n^{\log_2(5)}) = O(n^{2.32})$$

$$T_B(n) = 2 \cdot T(n - 1) + O(1) \Rightarrow T_B(n) = O(2^n)$$

$$T_C(n) = 9 \cdot T\left(\frac{n}{3}\right) + O(n^2) \Rightarrow T_C(n) = O(n^2 \cdot \log_2(n))$$

A escolha seria o algoritmo C, que funciona em tempo de execução $O(n^2 \log_2(n))$.

6. Dado $A[1, \dots, n]$, um vetor ordenado de inteiros distintos, você quer saber se existe um índice i para o qual $A[i] = i$. Dê um algoritmo de divisão-e-conquista que execute em tempo $O(\log n)$.

Solução: Tendo em vista que temos um vetor ordenado e esse é o princípio básico para o uso do algoritmo de busca binária, que de fato utiliza a técnica de dividir e conquistar, podemos usá-lo para a resolução de nosso problema. Com isso:

```
int binarySearch(int *A, int start, int end){
    int middle;
    if(start <= end){
        middle = (start + end)/2; // calcula a posição do meio do array
        if(middle == A[middle]) // checa se i = A[i]
            return middle;
        else if(middle < A[middle]) // log(n)
            binarySearch(A, start, middle-1); // busca na metade esquerda
        else // log(n)
            binarySearch(A, middle+1, end); // busca na metade direita
    }
    return -1; // elemento nao encontrado
}
```

7. Resolver o problema da Torre de Hanói quando se dispõe de 4 torres, em vez de 3. Determinar o número de movimentos de disco.

Solução: O funcionamento do problema da Torre de Hanói consiste em mover os discos de uma torre para outra, desde que obedeça duas regras básicas:

1. Só pode mover um disco por vez.
2. Um disco maior nunca pode ser colocado em cima de um disco menor.

Além disso, os discos da torre inicial estão dispostos em ordem decrescente, de baixo para cima e devem estar do mesmo modo na torre final. Originalmente, o problema ocorre com 3 torres: A, B e C, onde uma é de origem, outra de destino e outra auxiliar. A proposição do enunciado, no entanto, propõe a resolução do problema com 4 torres, sendo 2 delas auxiliares. Para a resolução do problema, temos:

```
void moveTower(int n, int orig, int tower);

void hanoiTower(int n, int orig, int dest, int aux1, int aux2){

    if(n == 0)
        return; // não há discos a ser movido

    if(n == 1){ // só há um disco a ser movido
        moveTower(n, orig, dest); // ele é movido pra torre final
        return;
    }

    // ocorre a recursao para mover n-2 discos
    hanoiTower(n-2, orig, aux1, aux2, dest);

    moveTower(n-1, orig, aux2);
    moveTower(n, orig, dest);
    moveTower(n-1, aux2, dest);

    // ocorre a recursao para mover n-2 discos da torre auxiliar para o destino
    hanoiTower(n-2, aux1, dest, orig, aux2);
}
```

Para $n = 0$, temos $T(0) = 0$;

Para $n = 1$, temos $T(1) = 1$, onde:

A função `moveTower()` é chamada uma única vez;

Para $n = 2$, temos $T(2) = 3$, onde:

A função `moveTower()` move o disco 1 da origem para a torre auxiliar 2;

A função `moveTower()` move o disco 2 da origem para o destino;

A função `moveTower()` move o disco 1 da torre 2 para o destino;

Para $n = 3$, temos $T(3) = 5$, onde:

A função `moveTower()` move o disco 1 para a torre auxiliar 1;
A função `moveTower()` move o disco 2 da origem para a torre auxiliar 2;
A função `moveTower()` move o disco 3 da origem para o destino;
A função `moveTower()` move o disco 2 da torre 2 para o destino;
A função `moveTower()` move o disco 1 da torre 1 para o destino;

Prosseguindo, temos que a função de recorrência é dada por $T(n) = 2T(n-2) + 3$, pois a função `hanoiTower()` é chamada 2 vezes recursivamente e a função chamada `moveTower()` 3 vezes. Através da técnica da expansão, temos:

$$T(n) = 2T(n-2) + 3$$

$$T(n) = 2(2T(n-4)+3) + 3$$

$$T(n) = 2(2(2T(n-6)+3)+3) + 3$$

$$T(n) = 8T(n-6) + 21$$

$$\vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots$$

$$T(n) = 2^k \cdot T(n - 2k) + (2^k \cdot 3) - 3$$

Onde, sabemos que $T(0) = 0$, logo $n = 2k = 0 \therefore k = n/2$, portanto:

$$T(n) = 2^{n/2} \cdot T(0) + (2^{n/2} \cdot 3) - 3 = 0 + (2^{n/2} \cdot 3) - 3 = (2^{n/2} \cdot 3) - 3$$

Com isso, concluímos que o algoritmo necessita de $(2^{n/2} \cdot 3) - 3$ movimentos de discos para resolver o problema da Torre de Hanoi com 4 torres.

8. **Explique a relação da técnica Divisão-e-Conquista com paralelismo. Como utilizar o paralelismo em um algoritmo divisão-e-conquista? Por que elas funcionam bem em conjunto?**

Solução: Algoritmos de divisão e conquista possuem grande capacidade de serem executados em sistemas multiprocessados e, portanto, são facilmente paralelizáveis. Essa capacidade se dá na fase de divisão em que os problemas são “quebrados” em partes menores visando diminuir a sua complexidade e atrelando a essa técnica uma característica natural de divisão do trabalho, no qual cada subproblema pode ser executado em um processador distinto dos demais.

9. **Dado um vetor A com n entradas, com cada entrada um número distinto. Sabe-se que a sequência de valores $A[1], A[2], \dots, A[n]$ é unimodal: ou seja, existe índice p entre 1 e n, os valores nas entradas do vetor aumentam até a posição p e, em seguida, reduzem, o resto do caminho, até que a posição**

n. Deseja-se encontrar o “ponto de máximo” p. Mostre como encontrar p através de um algoritmo $O(\log n)$.

Solução:

```
#include<vector>
#include<iostream>
using namespace std;

int find_num(int *arr, int b, int e){ //Array indexado 0 a n-1
    if(b==e) return b; // Um elemento
    else if(b==e-1) return arr[b]>arr[e] ? b : e; // 2 Elementos
    else{ // Mais de 2 elementos
        int m = (b+e)>>1; //Elemento do Meio

        if(arr[m]>arr[m-1] && arr[m]>arr[m+1]) return m; // Condição para máximo
        else if(arr[m]>arr[m+1]) return find_num(arr, b, m-1); // Segue para metade esquerda
        else if(arr[m]>arr[m-1]) return find_num(arr, m+1, e); // Segue para metade direita
    } // Complexidade  $O(\log n)$ 
    return -1; //Erro
}
```

10. Você está consultando para uma pequena empresa de investimentos. Eles estão fazendo uma simulação em que eles olham para n dias consecutivos de uma determinada ação, em algum momento no passado. Cara cada dia $i = 1, 2, \dots, n$; eles têm o preço p_i da ação neste dia. Suponha que durante este período de tempo, eles queriam comprar 1.000 ações em alguns dias e vender todas essas ações em algum dia (mais tarde). Eles querem saber: Quando eles deveriam ter vendido, a fim de maximizar os lucros? Por exemplo, suponha que $n = 3, p_1 = 9, p_2 = 1, p_3 = 5$. Veja que deveria retornar “comprar em 2, vender em 3” (compra no dia 2 e vender no dia 3 significa que eles teria feito 4 por ação, o máximo possível para esse período). Claramente, há um algoritmo simples que leva tempo $O(n^2)$: tentar todos os possíveis pares compra/venda e ver qual deles faz mais dinheiro. Elabore um algoritmo para encontrar os dias de compra e venda com tempo $O(n \log n)$.

Solução:

